EECE5644 — Machine Learning & Pattern Recognition

Homework Assignment 2: Solutions
Probability Models & Statistical Inference

Eduardo Scopelli Rivas

2026FALL

Part I — Random Variables, Joint Distributions, and Moments

Problem 1 Inverse-Transform Sampling: Uniform to Exponential

$X \sim U(0,1); Y = -\frac{ln(X)}{\lambda}, \lambda > 0$

(a) Using the CDF technique, derive the distribution of Y and show that Y ∼ Exponential(λ).

a) Distribution of Y

$$F_Y(y) = P(Y \leq y) = P\left(-\frac{\ln(x)}{\lambda} \leq y\right) = P\left(\ln(x) \geq -\lambda y\right) = P\left(X \geq e^{-\lambda y}\right)$$

$X \sim U(0,1) \rightarrow F_X(x) = x,\ x \in [0,1]$; $e^{-\lambda y} \in [0,1],\ y \geq 0$

$$F_Y(y) = P\left(X \geq e^{-\lambda y}\right) = 1 - e^{-\lambda y},\ y \geq 0 \longrightarrow \underline{f_Y(y) = \lambda e^{-\lambda y},\ y \geq 0}$$

$F_Y(y) = P(X \geq e^{-\lambda y}); F'_Y(y) = f_Y(y) = \lambda e^{-\lambda y}, y \geq 0$

(b) In Python or MATLAB, set $\lambda = 2$ and $seed = 42$. Draw $N = 100,000$ samples of X , transform them to obtain samples of Y , and report the sample mean and sample variance of Y . Compare these to the theoretical values $\frac{1}{\lambda}$ and $\frac{1}{\lambda^2}$.

```python
rng = np.random.default_rng(seed)

lam = 2
N = 100000

X = rng.uniform(0, 1, N)
Y = -np.log(X) / lam

print(f"{"Sample mean:": <18} {Y.mean(): .8f}; {"Sample var:": <16} {Y.var(): .8f}")

R_mean = 1 / lam
R_var = 1 / lam ** 2

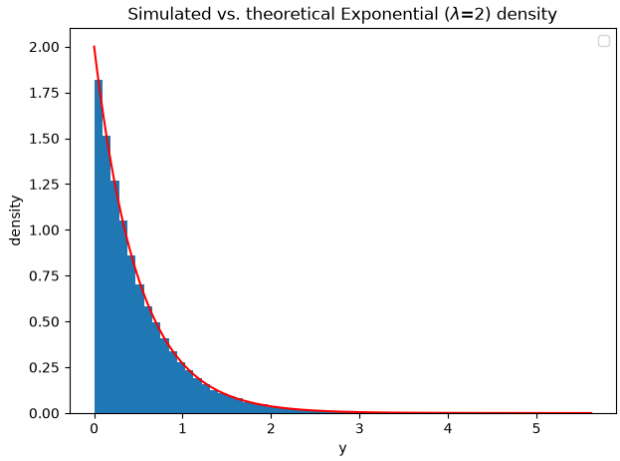
print(f"{"Theoretical mean:": <18} {R_mean: .8f}; {"Theoretical var:": <16} {R_var: .8f}")
```

```
Sample mean:       0.49964471;  Sample var:       0.25198023
Theoretical mean:  0.50000000;  Theoretical var:  0.25000000
```

(c) Plot a normalized histogram of your simulated Y samples overlaid with the theoretical Exponential(λ) density. Comment briefly on the agreement.

```python
plt.hist(Y, bins=60, density=True)
y_dist = np.linspace(0, Y.max(), 500)
plt.plot(y_dist, lam*np.exp(-lam*y_dist), color="red")

plt.xlabel("y")
plt.ylabel("density")
plt.title(f"Simulated vs. theoretical Exponential ($\\lambda$={lam}) density")
plt.legend()
plt.tight_layout()
plt.show()
```



This plot shows how closely the overlaid curve and the histogram follow each other, with a slight error noticeable near $y = 0$, mainly due to the simplification of the histogram (widths of each bar) when the curve presents a greater gradient. Additionally, such a deviation between the theoretical curve and the histogram based on samples will always be present due to the use of finite samples, which might approach the theoretical curve as N approaches infinity, but won't reach it outright.

Problem 2 Joint, Marginal, and Conditional Distributions of Two Discrete Random Variables

	$Y=0$	$Y=1$
$X=0$	0.25	0.05
$X=1$	0.10	0.20
$X=2$	0.15	0.25

(a) Compute the marginal pmfs $p_X(x)$ and $p_Y(y)$.

	$Y=0$	$Y=1$	$p_X(x)$
$X=0$	0.25	0.05	0.30
$X=1$	0.10	0.20	0.30
$X=2$	0.15	0.25	0.40
$p_Y(y)$	0.50	0.50	1.00

(b) Compute the conditional pmf $p_{Y|X}(y|X=1)$. Are X and Y statistically independent? Justify your answer using the joint and marginal pmfs (do not just assert it).

$$P_{Y|X}(y\,|\,X=1) = \frac{p(1,y)}{p(1)}$$

$$P_{Y|X}(\emptyset\,|\,X=1) = \frac{0.10}{0.30} = \frac{1}{3}$$

$$P_{Y|X}(1\,|\,X=1) = \frac{0.20}{0.30} = \frac{2}{3}$$

Independence: $p(x,y) = p_X(x)\,p_Y(y)$

Test 1: $X=Y=\emptyset$

$p(x,y) = 0.25$

$p_X(x)\,p_Y(y) = (0.30)(0.50) = 0.15$

$0.15 \neq 0.25$

Not independent

(c) Compute $E[X], V(X), E[Y], V(Y), E[XY], V(X,Y)$, and the correlation coefficient $\rho_{X,Y}$.

$\mathbb{E}[X] = \sum_x x\, p_X(x) = 0(0.30) + 1(0.30) + 2(0.40) = \underline{1.1}$

$\mathbb{V}[X] = \mathbb{E}[X^2] - \mathbb{E}[X]^2 = \left(0(0.30) + 1(0.30) + 2^2(0.40)\right) - 1.1^2 = 1.9 - 1.21 = \underline{0.69}$

$\mathbb{E}[Y] = \sum_y y\, p_Y(y) = 0(0.50) + 1(0.50) = \underline{0.50}$

$\mathbb{V}[Y] = \mathbb{E}[Y^2] - \mathbb{E}[Y]^2 = \left(0(0.50) + 1(0.50)\right) - 0.50^2 = 0.50 - 0.25 = \underline{0.25}$

$\mathbb{E}[XY] = \sum_{xy} xy\, p(x,y) = (1)(1)(0.20) + (2)(1)(0.25) = \underline{0.70}$

$\mathbb{V}[X,Y] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y] = 0.70 - 1.1 \cdot 0.50 = \underline{0.15}$

$\rho_{X,Y} = \frac{\mathbb{V}[X,Y]}{\sqrt{\mathbb{V}[X]\mathbb{V}[Y]}} = \frac{0.15}{\sqrt{0.69 \cdot 0.25}} \approx \underline{0.3612}$

(d) Verify your hand computations from (a)–(c) with a short script (10–15 lines) that encodes the table as a 2D array and recomputes all of the above quantities numerically.

```python
P = np.array([[0.25, 0.05],
              [0.10, 0.20],
              [0.15, 0.25]])
x_vals, y_vals = np.array([0,1,2]), np.array([0,1])

pX, pY = P.sum(axis=1), P.sum(axis=0)
pY_given_X1 = P[1,:] / pX[1] # Test X = 1
independent = np.allclose(np.outer(pX, pY), P)

EX = (x_vals*pX).sum()
VX = (x_vals**2*pX).sum() - EX**2
EY = (y_vals*pY).sum()
VY = (y_vals**2*pY).sum() - EY**2
EXY = sum(x_vals[i]*y_vals[j]*P[i,j] for i in range(3) for j in range(2))
VXY = EXY - EX*EY
rho = VXY / np.sqrt(VX*VY)

print("a) pX:", pX, " pY:", pY)
print("b) pY|X=1:", pY_given_X1, " Independence: ", independent)
print(f"c) E[X]={EX}, V(X)={VX}")
print(f" E[Y]={EY}, V(Y)={VY}")
print(f" E[XY]={EXY}, Cov(X,Y)={VXY}, rho={rho}")
```

```
a) pX: [0.3 0.3 0.4]  pY: [0.5 0.5]
b) pY|X=1: [0.33333333 0.66666667]  Independence:  False
c) E[X]=1.1, V(X)=0.69
   E[Y]=0.5, V(Y)=0.25
   E[XY]=0.7, Cov(X,Y)=0.1499999999999999, rho=0.3611575592573074
```

Problem 3 Covariance and Correlation from Real Data

(a) For jointly distributed random variables X, Y and constants $a, b, c, d \in R$, derive a general formula for $V(aX + bY, cX + dY)$ in terms of $V(X), V(Y)$, and $V(X, Y)$.

$$\mathbb{V}(aX+bY, cX+dY), \quad U = aX+bY, \; W = cX+dY \rightarrow \mathbb{V}(U,W) = \mathbb{E}\left[(U-\mathbb{E}U)(W-\mathbb{E}W)\right]$$

$$U-\mathbb{E}U = (aX+bY) - (a\mathbb{E}X + b\mathbb{E}Y) = a(X-\mathbb{E}X) + b(Y-\mathbb{E}Y) \; ; \; W-\mathbb{E}W = c(X-\mathbb{E}X) + d(Y-\mathbb{E}Y)$$

$$(U-\mathbb{E}U)(W-\mathbb{E}W) = \left[a(X-\mathbb{E}X) + b(Y-\mathbb{E}Y)\right]\left[c(X-\mathbb{E}X) + d(Y-\mathbb{E}Y)\right]$$

$$= ac(X-\mathbb{E}X)^2 + (ad+bc)(X-\mathbb{E}X)(Y-\mathbb{E}Y) + bd(Y-\mathbb{E}Y)^2$$

$$\mathbb{E}\left[(U-\mathbb{E}U)(W-\mathbb{E}W)\right] = ac\underbrace{\mathbb{E}\left[(X-\mathbb{E}X)^2\right]}_{\mathbb{V}(X)} + (ad+bc)\underbrace{\mathbb{E}\left[(X-\mathbb{E}X)(Y-\mathbb{E}Y)\right]}_{\mathbb{V}(X,Y)} + bd\underbrace{\mathbb{E}\left[(Y-\mathbb{E}Y)^2\right]}_{\mathbb{V}(Y)}$$

$$\mathbb{V}(aX+bY, cX+dY) = ac\mathbb{V}(X) + (ad+bc)\mathbb{V}(X,Y) + bd\mathbb{V}(Y)$$

(b) Load the sklearn.datasets.load_iris dataset. Using the sepal length and petal length features (columns 1 and 3, in cm) across all 150 samples, compute: the empirical mean vector, the 2×2 sample covariance matrix, and the sample correlation coefficient $\hat{\rho}$.

```
data = load_iris()
X = data.data[:, 0]
Y = data.data[:, 2]

mean_vec = np.array([X.mean(), Y.mean()])
cov = np.cov(X, Y, ddof=1)
rho = cov[0,1] / np.sqrt(cov[0,0]*cov[1,1])

print("mean vector:", mean_vec)
print("covariance matrix:\n", cov)
print("correlation rho:", rho)
```

```
mean vector: [5.84333333 3.758    ]
covariance matrix:
 [[0.68569351 1.27431544]
 [1.27431544 3.11627785]]
correlation rho: 0.8717537758865832
```

(c) Using your formula from (a), compute $V(2X-Y, X+3Y)$ for X = sepal length, Y = petal length, plugging in the empirical moments from (b). Verify your result by directly computing the sample covariance of the transformed variables $2X-Y$ and $X+3Y$ in code.

```
a, b, c, d = 2, -1, 1, 3
VX, VY, CXY = cov[0,0], cov[1,1], cov[0,1]
V_formula = a*c*VX + b*d*VY + (a*d + b*c)*CXY
```

```
4
5  U = a*X + b*Y # 2X - Y
6  W = c*X + d*Y # X + 3Y
7  V_direct = np.cov(U, W, ddof=1)[0,1]
8
9  print("V(2X-Y, X+3Y) calculated :", V_formula)
10 print("V(2X-Y, X+3Y) direct :", V_direct)
```

```
V(2X-Y, X+3Y) calculated : -1.6058693512304325
V(2X-Y, X+3Y) direct     : -1.605869351230424
```

Part II — Random Vectors and the Multivariate Normal

Problem 4 Random Vectors and Linear Transformations

$$X \in \mathbb{R}^{3x1}, \mu_X = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \sigma_X = \begin{bmatrix} 1 & 1.8 & 2.43 \\ 1.8 & 4 & 5.4 \\ 2.43 & 5.4 & 9 \end{bmatrix}$$

Using a linear mapping $Y = AX + b$ we want to obtain another vector Y so that:

$$Y \in \mathbb{R}^{3x1}, \mu_Y = \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix}, \Sigma_Y = \begin{bmatrix} 9 & -5.4 & -2.43 \\ -5.4 & 4 & 1.8 \\ -2.43 & 1.8 & 1 \end{bmatrix}$$

(a) First, using $Z = A_1X + b_1$, transform X into a 3 × 1 random vector Z with $\mu_Z = 0$ and $\sigma_Z = I$. Determine the matrix A_1 and the vector b_1.

```
muX = np.array([1., 2., 3.])
SigX = np.array([[1, 1.8, 2.43],
                 [1.8, 4, 5.4],
                 [2.43, 5.4, 9.]])

SigX_sqrt = linalg.sqrtm(SigX).real
A1 = np.linalg.inv(SigX_sqrt)
b1 = -A1 @ muX
print("A1:\n", A1)
print("b1: ", b1, "\n")

muZ = A1 @ muX + b1
SigZ = A1 @ SigX @ A1.T
print("muZ:", muZ)
print("SigZ:\n", SigZ)
```

```
A1:
 [[ 2.18346522 -0.69475997 -0.11378095]
 [-0.69475997  1.30024694 -0.45633916]
 [-0.11378095 -0.45633916  0.60299568]]
b1:  [-0.45260242 -0.53671643 -0.78252776]

muZ: [0. 0. 0.]
SigZ:
 [[ 1.00000000e+00  2.91691098e-15  9.82787632e-16]
 [ 3.38856326e-15  1.00000000e+00 -1.49035388e-15]
 [ 9.77815218e-16 -2.08814071e-15  1.00000000e+00]]
```

(b) Second, using $Y = A_2Z + b_2$, transform Z into Y so that Y has the prescribed mean and covariance given in (2). Determine the matrix A_2 and the vector b_2.

```
muY = np.array([3., 2., 1.])
SigY = np.array([[9, -5.4, -2.43],
                 [-5.4, 4, 1.8],
                 [-2.43, 1.8, 1.]])

```

```
A2 = linalg.sqrtm(SigY).real
b2 = muY.copy()
print("A2:\n", A2)
print("b2: ", b2, "\n")

muY_check = A2 @ muZ + b2
SigY_check = A2 @ SigZ @ A2.T
print("muY check:", muY_check)
print("SigY check:\n", SigY_check)
```

```
A2:
 [[ 2.68624191 -1.2260143  -0.53008805]
 [-1.2260143   1.48618834  0.53678036]
 [-0.53008805  0.53678036  0.65640955]]
b2:  [3. 2. 1.]

muY check: [3. 2. 1.]
SigY check:
 [[ 9.   -5.4  -2.43]
 [-5.4   4.    1.8 ]
 [-2.43  1.8   1.  ]]
```

(c) Finally, combine the above two mappings to obtain the overall transformation matrix A and vector b. Verify that the transformation produces the required mean and the covariance given in (2).

```
A = A2 @ A1
b = A2 @ b1 + b2
print("A =\n", A)
print("b =", b, "\n")

muY_final = A @ muX + b
SigY_final = A @ SigX @ A.T
print("Final muY:", muY_final)
print("Final SigY:\n", SigY_final)
```

```
A =
 [[ 6.77741537 -3.21851475 -0.06580564]
 [-3.77057913  2.5392436  -0.21503263]
 [-1.60504924  0.76668559  0.21117215]]
b = [2.85703105 1.33718981 0.4381616 ]

Final muY: [3. 2. 1.]
Final SigY:
 [[ 9.   -5.4  -2.43]
 [-5.4   4.    1.8 ]
 [-2.43  1.8   1.  ]]
```

Problem 5 Conditional Distributions of a Multivariate Gaussian

Let $V = (X_1, X_2, X_3)^T \sim N(\mu, \Sigma)$ with:

$$\mu = 0, \Sigma = \begin{bmatrix} 4 & 2 & 1 \\ 2 & 3 & 0.5 \\ 1 & 0.5 & 2 \end{bmatrix}$$

Partition V into $X_1 = (X_1, X_2)^2$ and $X_2 = X_3$.

(a) State the general Schur-complement formula for the conditional distribution $X1|X2 \sim N(\mu_{1|2}, \Sigma_{1|2})$ of a jointly Gaussian random vector, and briefly explain, in one or two sentences, why $\Sigma_{1|2}$ does not depend on the value of X_2 that is conditioned on.

a) $V = \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix} \sim N(\mu, \Sigma) \rightarrow \mu = 0, \Sigma = \begin{bmatrix} 4 & 2 & 1 \\ 2 & 3 & 0.5 \\ 1 & 0.5 & 2 \end{bmatrix} \Rightarrow V \begin{cases} X_1 = (X_1, X_2)^T \\ X_2 = X_3 \end{cases} \Rightarrow V = \begin{bmatrix} X_1 \\ X_2 \end{bmatrix} \sim N\left(\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix}\right)$

$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$, $\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$: Schur-complement of Σ_{22} in Σ

$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$$

$$\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$$

$\Sigma_{1|2}$ does not depend on the value of X_2 due to the nature of a Gaussian distribution, which causes the shape of the conditional distribution to stay the same no matter the slice $X_2 = x_2$, with each shift only affecting the location / mean of the slice but maintaining the shape. Algebraically, looking at the formulas above, the value of x_2 isn't in the formula for $\Sigma_{1|2}$, it only affects the formula for the conditional mean $\mu_{1|2}$.

(b) Compute $\mu_{1|2}$ and $\Sigma_{1|2}$ numerically for $X_3 = 1.5$.

b) $\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $\Sigma = \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix} & \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} \\ \begin{bmatrix} 1 & 0.5 \end{bmatrix} & \begin{bmatrix} 2 \end{bmatrix} \end{bmatrix}$, $X_3 = 1.5$

$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_1) = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}\left(\frac{1}{2}\right)(1.5 - 0) = \frac{3}{4}\begin{bmatrix} 1 \\ 0.5 \end{bmatrix} = \underline{\begin{bmatrix} 0.75 \\ 0.325 \end{bmatrix}}$

$\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21} = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix} - \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}\left(\frac{1}{2}\right)\begin{bmatrix} 1 & 0.5 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix} - \frac{1}{2}\begin{bmatrix} 1 & 0.5 \\ 0.5 & 0.25 \end{bmatrix} = \underline{\begin{bmatrix} 3.5 & 1.75 \\ 1.75 & 2.875 \end{bmatrix}}$

(c) Verify your closed-form answer from (b) empirically: draw a very large number ($N \geq 2 \times 10^6$, seed=42) of samples of V, keep only those with X_3 within a small tolerance band of 1.5 (e.g. $|X_3 - 1.5| < 0.03$), and compute the empirical mean and covariance of (X_1, X_2) within that band. Report how closely it matches (b).

```
mu = np.zeros(3)
Sigma = np.array([[4, 2, 1],
                  [2, 3, 0.5],
                  [1, 0.5, 2]])

N = int(2e6)
V = rng.multivariate_normal(mu, Sigma, size=N)

tolerance = 0.03
mask = np.abs(V[:, 2] - 1.5) < tolerance
print("samples retained:", mask.sum(), "\n")

emp_mean = V[mask, :2].mean(axis=0)
emp_cov = np.cov(V[mask, 0], V[mask, 1], ddof=1)
cf_mean = np.array([0.75, 0.375])
cf_cov = np.array([[3.5, 1.75],
                   [1.75, 2.875]])

print("Empirical mean: ", emp_mean)
print("Closed-form mean:", cf_mean,"\n")

print("Empirical cov:\n", emp_cov)
print("Closed-form cov:\n", cf_cov)
```

```
samples retained: 19428

Empirical mean:   [0.77212229 0.39436917]
Closed-form mean: [0.75  0.375]

Empirical cov:
 [[3.53784785 1.75840931]
 [1.75840931 2.87513737]]
Closed-form cov:
 [[3.5   1.75 ]
 [1.75  2.875]]
```

Problem 6 Sums and Products of Independent Gaussian Random Variables

Let $X \sim N(\mu_1, \sigma_1^2)$ and $Y \sim N(\mu_2, \sigma_2^2)$

(a) Using either the convolution integral of the two densities or moment generating functions, show that:

$Z = X + Y \sim N(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$

a) $Z = X + Y$

MGF of $N(\mu, \sigma^2) \rightarrow M(t) = \exp\left(\mu t + \frac{1}{2}\sigma^2 t^2\right) \rightarrow M_Z(t) = M_{X+Y}(t) = \mathbb{E}\left[e^{t(X+Y)}\right] = \mathbb{E}\left[e^{tX} \cdot e^{tY}\right] = \mathbb{E}\left[e^{tX}\right] \cdot \mathbb{E}\left[e^{tY}\right] = M_X(t) M_Y(t)$

$$M_X(t) = \int_{-\infty}^{\infty} e^{tx} \cdot \frac{1}{\sigma\sqrt{2\pi}} e^{\frac{-(x-\mu)^2}{2\sigma^2}} dx \rightarrow tx - \frac{(x-\mu)^2}{2\sigma^2} = tx - \frac{x^2}{2\sigma^2} + \frac{\mu x}{\sigma^2} - \frac{\mu^2}{2\sigma^2} = -\frac{1}{2\sigma^2}x^2 + \left(t + \frac{\mu}{\sigma^2}\right)x - \frac{\mu^2}{2\sigma^2}$$

$$M_X(t) = \exp\left[\frac{\sigma^2\left(t + \frac{\mu}{\sigma^2}\right)^2}{2} - \frac{\mu^2}{2\sigma^2}\right] = \exp\left[\frac{1}{2}\sigma^2 t^2 + \mu t + \cancel{\frac{\mu^2}{2\sigma^2}} - \cancel{\frac{\mu^2}{2\sigma^2}}\right]$$

$$M_X(t) = \exp\left[\mu t + \frac{1}{2}\sigma^2 t^2\right]$$

$$\vdots$$

$$M_Z(t) = M_X(t) M_Y(t) = \exp\left(\mu_1 t + \frac{1}{2}\sigma_1^2 t^2\right) \cdot \exp\left(\mu_2 t + \frac{1}{2}\sigma_2^2 t^2\right)$$

$$M_Z(t) = \exp\left((\mu_1 + \mu_2)t + \frac{1}{2}(\sigma_1^2 + \sigma_2^2)t^2\right) \rightarrow \text{MGF of } N\left(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2\right)$$

(b) Consider instead the product of the two densities viewed as functions of a single variable x (not a sum of random variables): $p(x) = N(x; \mu_1, \sigma_1^2) * N(x; \mu_2, \sigma_2^2)$. By completing the square in the exponent, show that $p(x)$ is proportional to a Gaussian density $N(x; \mu_\star, \sigma_\star^2)$ and derive $\mu_\star$ and $\sigma_\star^2$ in terms of $\mu_1, \mu_2, \sigma_1^2, \sigma_2^2$. (This identity is the computational core of Bayesian updating with a Gaussian likelihood and Gaussian prior, e.g. in a Kalman filter measurement update).

b) $p(x) = N\left(x; \mu_1, \sigma_1^2\right) \cdot N\left(x; \mu_2, \sigma_2^2\right)$

$$p(x) \propto \exp\left[-\frac{(x-\mu_1)^2}{2\sigma_1^2} - \frac{(x-\mu_2)^2}{2\sigma_2^2}\right]$$

$$\downarrow$$

$$\frac{-x^2}{2\sigma_1^2} + \frac{\mu_1 x}{\sigma_1^2} - \frac{\mu_1^2}{2\sigma_1^2} - \frac{x^2}{2\sigma_2^2} + \frac{\mu_2 x}{\sigma_2^2} - \frac{\mu_2^2}{2\sigma_2^2} = \underbrace{-\frac{1}{2}\left(\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}\right)}_{\gamma} x^2 + \underbrace{\left(\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2}\right)}_{\beta} x - \underbrace{\left(\frac{\mu_1^2}{2\sigma_1^2} + \frac{\mu_2^2}{2\sigma_2^2}\right)}_{C}$$

$$-\frac{\gamma}{2}x^2 + \beta x - C \rightarrow -\frac{\gamma}{2}x^2 + \beta x = -\frac{\gamma}{2}\left(x - \frac{\beta}{\gamma}\right)^2 + \frac{\beta^2}{2\gamma} \rightarrow \frac{\beta^2}{2\gamma} - C \text{ is constant, can be ignored for proportionality.}$$

$$p(x) \propto \exp\left[-\frac{\gamma}{2}\left(x - \frac{\beta}{\gamma}\right)^2\right] \Rightarrow \mu_* = \frac{\beta}{\gamma}, \quad \sigma_*^2 = \frac{1}{\gamma}$$

$$\mu_* = \frac{\beta}{\gamma} = \frac{\mu_1\sigma_2^2 + \mu_2\sigma_1^2}{\sigma_1^2 + \sigma_2^2} \qquad \sigma_*^2 = \frac{1}{\gamma} = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}$$

(c) Let $\mu_1 = 2, \sigma_1 = 1$ and $\mu_2 = -1, \sigma_2 = 2$. Using seed = 42, simulate $N = 200,000$ independent samples of X and Y, form $Z = X + Y$, and report the empirical mean and variance of Z against the theoretical values from (a). Also evaluate $\mu_\star, \sigma_\star^2$ from (b) numerically for these parameter values.

```
mu1, s1 = 2, 1
mu2, s2 = -1, 2
var1, var2 = s1**2, s2**2
N = 200_000

X = rng.normal(mu1, s1, N)
```

```
Y = rng.normal(mu2, s2, N)
Z = X + Y

print("empirical mean Z:", Z.mean(), " theory:", mu1+mu2)
print("empirical var Z:", Z.var(), " theory:", var1+var2)

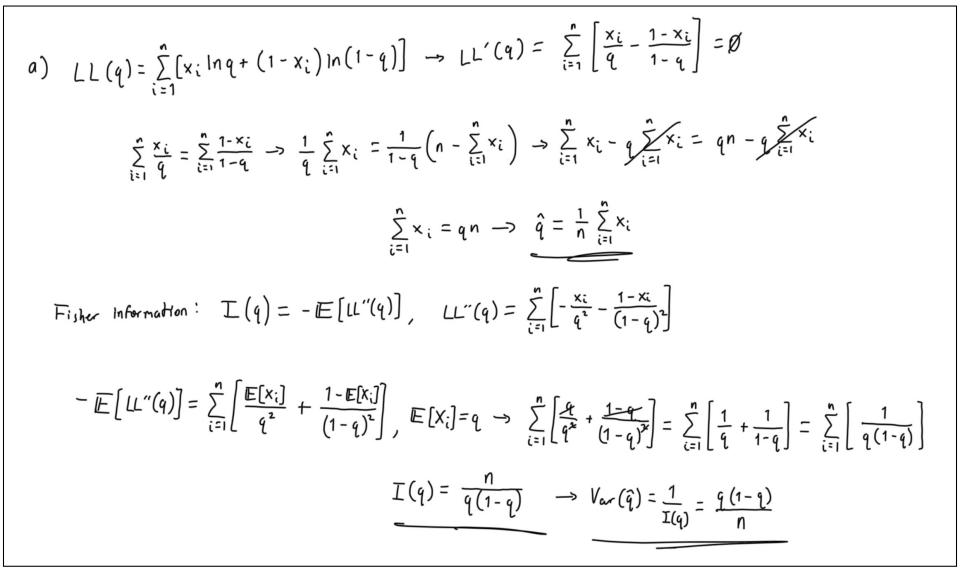
mu_star = (mu1*var2 + mu2*var1) / (var1+var2)
var_star = (var1*var2) / (var1+var2)
print("mu_star:", mu_star, " var_star:", var_star)
```

```
empirical mean Z: 0.9980188048909078  theory: 1
empirical var  Z: 4.985278856543919  theory: 5
mu_star: 1.4  var_star: 0.8
```

Part III — Useful Distributions, Inference, and Simulation

Problem 7 Bernoulli / Binomial Estimation on Real Data

(a) For i.i.d. $X_1, ..., X_n \sim$ Bernoulli(q), derive the maximum-likelihood estimator $\hat{q}$, and derive the Fisher information $I(q)$ and the resulting asymptotic variance of $\hat{q}$.



(b) Load the Wisconsin Diagnostic Breast Cancer (WDBC) dataset. Define $X_i = 1$ if sample i is malignant and $X_i = 0$ otherwise, and compute the MLE $\hat{q}$ (the malignancy rate) over all $n = 569$ samples.

```
data = load_breast_cancer()
print("target_names:", data.target_names)

malignant = (data.target == 0).astype(int)
n = len(malignant)
qhat = malignant.mean()
print("n =", n, " qhat (malignancy rate) =", qhat)
```

```
target_names: ['malignant' 'benign']
n = 569  qhat (malignancy rate) = 0.37258347978910367
```

(c) Construct a 95% Wald confidence interval for q using your formula from (a), and also construct a 95% Wilson score interval. Report both and briefly comment on why they might differ.

```
z = stats.norm.ppf(0.975)
print("z: ", z)

```

```
# Wald
wald_hw = z * np.sqrt(qhat*(1-qhat)/n)
wald_ci = (qhat - wald_hw, qhat + wald_hw)

# Wilson score
denom = 1 + z**2/n
center = (qhat + z**2/(2*n)) / denom
wilson_hw = (z/denom) * np.sqrt(qhat*(1-qhat)/n + z**2/(4*n**2))
wilson_ci = (center - wilson_hw, center + wilson_hw)

print("Wald 95% CI: ", wald_ci)
print("Wilson 95% CI:", wilson_ci)
```

```
z:  1.959963984540054
Wald 95% CI:   (np.float64(0.33285684938465654), np.float64(0.4123101101935508))
Wilson 95% CI: (np.float64(0.333835509125589), np.float64(0.41304035370300096))
```

The two intervals are close here because the sample size is reasonably large and the value of $\hat{q} \simeq 0.37$ isn't close to the boundaries of 0 or 1. This gap would increase noticeably for smaller sample sizes or a closer value of $\hat{q}$ to one of the boundaries.

(d) Using seed = 42, simulate $100,000$ draws from Binomial$(n, \hat{q})$ and compare the simulated mean and variance to the theoretical $n\hat{q}$ and $n\hat{q}(1-\hat{q})$.

```
N = 100000

sims = rng.binomial(n, qhat, N)
print(f"simulated mean: {sims.mean():.8f}; Theory n*qhat: {n*qhat:.8f}")
print(f"simulated var : {sims.var():.8f}; theory n*qhat*(1-qhat):{n*qhat*(1-qhat):.8f}")
```

```
simulated mean: 212.05187000; Theory n*qhat: 212.00000000
simulated var : 133.22093950; theory n*qhat*(1-qhat):133.01230228
```

Problem 8 Categorical / Multinomial Estimation and Dirichlet Conjugacy

(a) For i.i.d. categorical observations over K classes with unknown probability vector $\pi = (\pi_1, ..., \pi_K)$, derive the MLE $\hat{\pi}_k = \frac{n_k}{n}$ (where n_k is the count of class k), subject to $\Sigma_k \pi_k = 1$ (use a Lagrange multiplier).

a) $LL(\pi) = \sum_{k=1}^{K} n_k \ln(\pi_k)$; $g(\pi) = \sum_k \pi_k - 1 = 0$

$$\mathcal{L}(\pi, \lambda) = \sum_{k=1}^{K} n_k \ln \pi_k - \lambda\left(\sum_{k=1}^{K} \pi_k - 1\right)$$

$$\mathcal{L}(\pi_k, \lambda) = n_k \ln \pi_k - \lambda(\pi_k - 1) \rightarrow \frac{\partial \mathcal{L}}{\partial \pi_k} = \frac{n_k}{\pi_k} - \lambda = 0 \rightarrow \pi_k = \frac{n_k}{\lambda} \rightarrow \sum_{k=1}^{K} \pi_k = \frac{1}{\lambda}\sum_{k=1}^{K} n_k = 1 = \frac{n}{\lambda}$$

$$\lambda = n \rightarrow \hat{\pi}_k = \frac{n_k}{\lambda}, \text{ substitute } \lambda: \hat{\pi}_k = \frac{n_k}{n}$$

(b) Suppose $\pi \sim$ Dirichlet(α) a priori and we observe class counts $n = (n_1, ..., n_K)$. Derive the posterior distribution of π and show it is Dirichlet($\alpha + n$) (Dirichlet–Multinomial conjugacy).

b) $\pi \sim \text{Dirichlet}(\alpha)$, $n = (n_1, \ldots, n_K)$

$$p(\pi) \propto \prod_{k=1}^{K} \pi_k^{\alpha_k - 1} \; ; \quad p(\pi \mid n) = \frac{p(n \mid \pi)\, p(\pi)}{p(n)} \; ; \quad p(n \mid \pi) \propto \prod_{k=1}^{K} \pi_k^{n_k}$$

$$p(\pi \mid n) \propto p(n \mid \pi) p(\pi) = \prod_{k=1}^{K} \pi_k^{n_k} \cdot \prod_{k=1}^{K} \pi_k^{\alpha_k - 1} = \prod_{k=1}^{K} \pi^{\alpha_k + n_k - 1} \rightarrow \text{Dirichlet}(\alpha + n)$$

(c) Load iris dataset used in Problem-3. Using seed=42, draw a random subsample of 90 of the 150 rows without replacement and tabulate the class counts n_1, n_2, n_3 in the subsample.

```
data = load_iris()
y = data.target

idx = rng.choice(len(y), size=90, replace=False)
sub_y = y[idx]
counts = np.array([np.sum(sub_y == k) for k in range(3)])
print("subsample class counts:", counts, "\n")
```

```
subsample class counts: [28 31 31]
```

(d) Compute the MLE $\hat{\pi}$ from (c). Then, using a Dirichlet$(1, 1, 1)$ prior, compute the posterior mean of π using your result from (b). Compare the MLE and the posterior mean, and explain briefly (one to two sentences) why they differ the way they do.

```
n = counts.sum()
mle = counts / n
print("MLE pihat:", mle)

alpha_prior = np.array([1, 1, 1])
post_alpha = alpha_prior + counts
print("posterior Dirichlet params:", post_alpha)

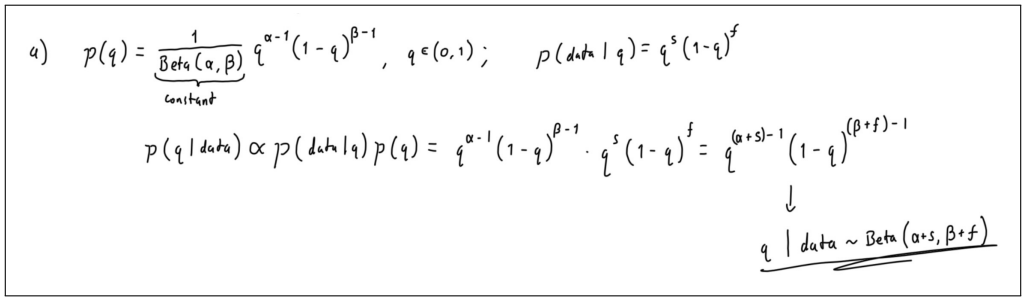
post_mean = post_alpha / post_alpha.sum()
print("posterior mean:", post_mean)
```

```
MLE pihat: [0.33333333 0.34444444 0.32222222]
posterior Dirichlet params: [31 32 30]
posterior mean: [0.33333333 0.34408602 0.32258065]
```

The reason for the small differences between the MLE $\hat{\pi}$ and the posterior mean occurs due to the prior acting like a few extra observations added to each class, which only slightly pulls the estimate towards a uniform distribution, but due to the relatively large sample data only causes a minimal change.

Problem 9 Beta-Bernoulli Sequential Bayesian Updating

(a) Show that the Beta(α, β) distribution is a conjugate prior for a Bernoulli likelihood: if $q \sim$ Beta(α, β) and we observe s successes and f failures in $s + f$ i.i.d. Bernoulli(q) trials, derive the posterior (q — data) $\sim$ Beta($\alpha + s, \beta + f$).



(b) Using seed=42, simulate N = 50 flips of a coin with true bias $q = 0.7$. Starting from a Beta(1, 1) (uniform) prior, update the posterior sequentially, one flip at a time, using your result from (a).

```
N = 50
q_true = 0.7
flips = (rng.uniform(0, 1, N) < q_true).astype(int)

alpha0, beta0 = 1, 1
a, b = alpha0, beta0
means, ci_lo, ci_hi = [], [], []

for x in flips:
    if x == 1:
        a += 1
    else:
        b += 1
    means.append(a / (a+b))
    lo, hi = stats.beta.ppf([0.05, 0.95], a, b)
    ci_lo.append(lo)
    ci_hi.append(hi)

print("successes:", flips.sum(), " failures:", N - flips.sum())
print("final (alpha, beta):", (a, b))
```

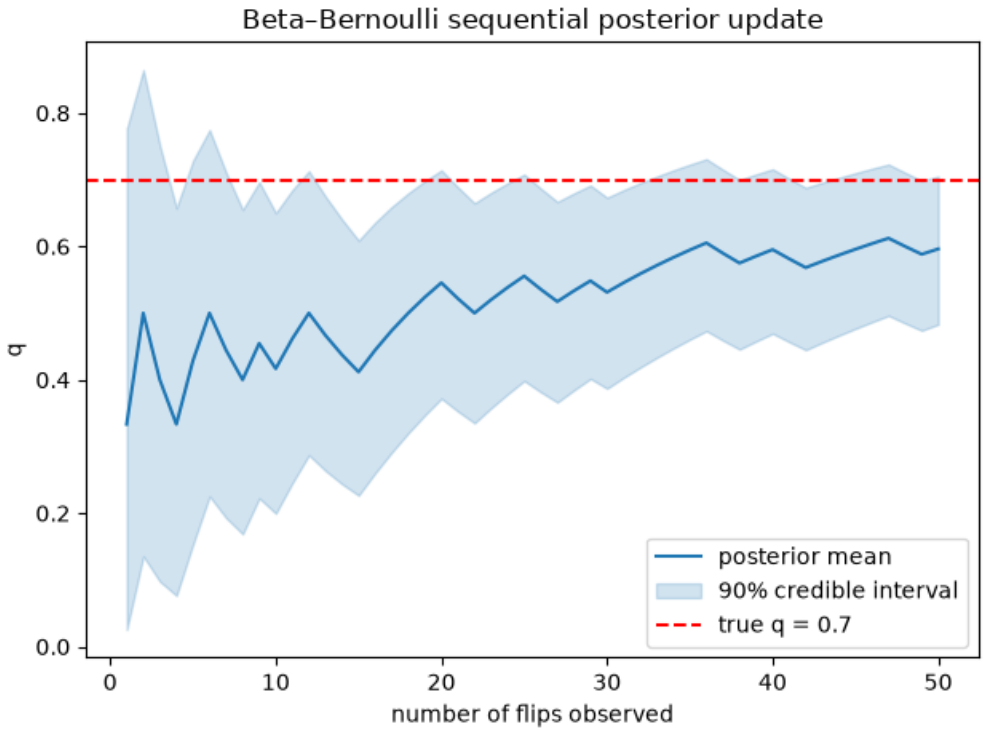
```
successes: 30  failures: 20
final (alpha, beta): (31, 21)
```

(c) Plot the posterior mean of p and a 90% credible interval (e.g. using the Beta quantile function) as a function of the number of flips observed so far, together with a horizontal reference line at the true value $q = 0.7$.

```
n_flips = np.arange(1, N+1)

plt.plot(n_flips, means, color='C0', label="posterior mean")
plt.fill_between(n_flips, ci_lo, ci_hi, color='C0', alpha=0.2, label="90% credible interval")
```

```
plt.axhline(q_true, color='r', linestyle='--', label=f"true q = {q_true}")
plt.xlabel("number of flips observed")
plt.ylabel("q")
plt.title("Beta-Bernoulli sequential posterior update")
plt.legend()
plt.tight_layout()
plt.savefig("p9_sequential_update.png", dpi=150)
plt.show()
```



(d) Report the final posterior parameters $(\alpha_{50}, \beta_{50})$, the final posterior mean, and the final 90% credible interval. Comment on whether the interval contains the true q.

```
final_mean = a / (a+b)
final_lo, final_hi = stats.beta.ppf([0.05, 0.95], a, b)
print("final posterior mean:", final_mean)
print("final 90% credible interval:", (final_lo, final_hi))
print("contains true q?", final_lo <= q_true <= final_hi)
```

```
final posterior mean: 0.5961538461538461
final 90% credible interval: (np.float64(0.4830716512438429), np.float64(0.7049888508764157))
contains true q? True
```

As shown in the image above, the interval $(0.4831, 0.7050)$ contains the true value of $q = 0.7$, with the posterior mean value of 0.5962 undershooting the true value by a bit. This difference can be attributed to the small sample size of 50 flips, however, the credible interval is still wide enough to still capture the obtained value.

Problem 10 Monte Carlo Simulation of a Gaussian Mixture Model

$Z \in \{1,2\}, P(Z=1) = w_1 = 0.4, P(Z=2) = w_2 = 0.6, X|Z=k \sim N(\mu_k, \Sigma_k)$

$$\mu_1 = \begin{bmatrix}0\\0\end{bmatrix}, \Sigma_1 = I_2, \mu_2 = \begin{bmatrix}4\\3\end{bmatrix}, \Sigma_2 = \begin{bmatrix}2 & 0.5\\0.5 & 1\end{bmatrix}$$

(a) Using the law of total expectation and the law of total variance, derive general formulas for the overall mean $E[X]$ and covariance $V(X)$ of a K-component GMM with weights w_k , means μ_k, and covariances Σ_k, in terms of the overall mean $\mu = \Sigma_k w_k \mu_k$. (Your covariance formula should have a "within-component" term and a "between-component" term.)

a) $Z \in \{1,2\}$, $P(Z=1) = w_1 = 0.4$, $P(Z=2) = w_2 = 0.6$; $X|Z=k \sim N(\mu_k, \Sigma_k)$

$\mu_1 = \begin{bmatrix}0\\0\end{bmatrix}$, $\Sigma_1 = I_2$, $\mu_2 = \begin{bmatrix}4\\3\end{bmatrix}$, $\Sigma_2 = \begin{bmatrix}2 & 0.5\\0.5 & 1\end{bmatrix}$

Total expectation: $\mathbb{E}[X] = \mathbb{E}_Z[\mathbb{E}[X|Z]] = \sum_k w_k \mathbb{E}[X|Z=k] = \sum_k w_k \mu_k = \mu$

Total variance: $\mathbb{V}[X] = \mathbb{E}_Z[\mathbb{V}[X|Z]] + \mathbb{V}_Z[\mathbb{E}[X|Z]]$; $\mathbb{E}_Z[\mathbb{V}[X|Z]] = \sum_k w_k \Sigma_k$; $\mathbb{V}_Z[\mathbb{E}[X|Z]] = \sum_k w_k (\mu_k - \mu)(\mu_k - \mu)^T$

$$\mathbb{V}[X] = \sum_k w_k \Sigma_k + \sum_k w_k (\mu_k - \mu)(\mu_k - \mu)^T$$

(b) Evaluate your formulas from (a) numerically for the GMM given above.

b) $\mu = 0.4\begin{bmatrix}0\\0\end{bmatrix} + 0.6\begin{bmatrix}4\\3\end{bmatrix} = \begin{bmatrix}2.4\\1.8\end{bmatrix}$; $\sum_k w_k \Sigma_k = 0.4\begin{bmatrix}1 & 0\\0 & 1\end{bmatrix} + 0.6\begin{bmatrix}2 & 0.5\\0.5 & 1\end{bmatrix} = \begin{bmatrix}1.6 & 0.3\\0.3 & 1\end{bmatrix}$

$\mu_1 - \mu = \begin{bmatrix}-2.4\\-1.6\end{bmatrix} \rightarrow (\mu_1 - \mu)(\mu_1 - \mu)^T = \begin{bmatrix}5.76 & 4.32\\4.32 & 3.24\end{bmatrix}$; $\mu_2 - \mu = \begin{bmatrix}1.6\\1.2\end{bmatrix} \rightarrow (\mu_2 - \mu)(\mu_2 - \mu)^T = \begin{bmatrix}2.56 & 1.92\\1.92 & 1.44\end{bmatrix}$

$\sum_k w_k (\mu_k - \mu)(\mu_k - \mu)^T = 0.4\begin{bmatrix}5.76 & 4.32\\4.32 & 3.24\end{bmatrix} + 0.6\begin{bmatrix}2.56 & 1.92\\1.92 & 1.44\end{bmatrix} = \begin{bmatrix}3.84 & 2.88\\2.88 & 2.16\end{bmatrix}$

$$\mathbb{V}[X] = \begin{bmatrix}1.6 & 0.3\\0.3 & 1\end{bmatrix} + \begin{bmatrix}3.84 & 2.88\\2.88 & 2.16\end{bmatrix} = \begin{bmatrix}5.44 & 3.18\\3.18 & 3.16\end{bmatrix}$$

```python
w = np.array([0.4, 0.6])
mus = [np.array([0., 0.]), np.array([4., 3.])]
Sigs = [np.eye(2), np.array([[2, 0.5], [0.5, 1.]])]

mu = sum(w[k]*mus[k] for k in range(2))
within = sum(w[k]*Sigs[k] for k in range(2))
between = sum(w[k]*np.outer(mus[k]-mu, mus[k]-mu) for k in range(2))
cov_overall = within + between

print("overall mean (mu):", mu)
print("within-component term:\n", within)
print("between-component term:\n", between, "\n")
print("total covariance:\n", cov_overall)
```

```
overall mean (mu): [2.4 1.8]
within-component term:
 [[1.6 0.3]
 [0.3 1. ]]
between-component term:
 [[3.84 2.88]
 [2.88 2.16]]

total covariance:
 [[5.44 3.18]
 [3.18 3.16]]
```

(c) Using seed = 42, simulate N = 5000 samples from this GMM (first sample Z_i , then $X_i|Z_i$), and report the empirical mean and covariance of the samples. Compare to (b).

```python
N = 5000
Z = rng.choice([0, 1], size=N, p=w)
X = np.zeros((N, 2))
for k in range(2):
    idx = Z == k
    X[idx] = rng.multivariate_normal(mus[k], Sigs[k], size=idx.sum())

emp_mean = X.mean(axis=0)
emp_cov = np.cov(X[:,0], X[:,1], ddof=1)
print("empirical mean:", emp_mean)
print("empirical cov:\n", emp_cov)
```

```
empirical mean: [2.40134192 1.83247211]
empirical cov:
 [[5.45035015 3.19041742]
 [3.19041742 3.15810538]]
```

(d) Produce a scatter plot of your simulated GMM samples, colored by which component generated each point. For contrast, also generate N = 1000 samples from a two-moons dataset. Plot the GMM samples and the two-moons samples side by side, and in one or two sentences explain why a single mean vector and covariance matrix would be a poor summary of the two-moons data even though such a summary is meaningful for the GMM.

```python
Xm, ym = make_moons(n_samples=1000, noise=0.1, random_state=42)

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

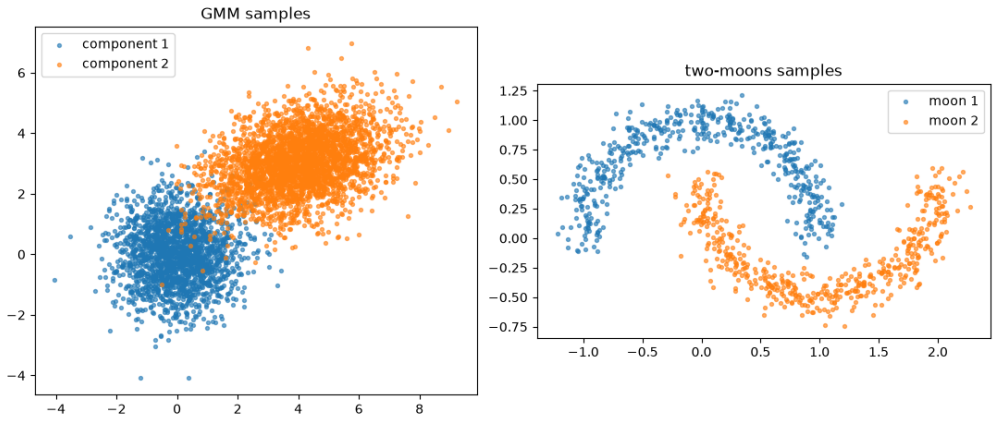
for k in range(2):
    axes[0].scatter(X[Z==k, 0], X[Z==k, 1], s=8, alpha=0.6, label=f"component {k+1}")

axes[0].set_title("GMM samples")
axes[0].legend(); axes[0].set_aspect('equal')

for k in range(2):
    axes[1].scatter(Xm[ym==k, 0], Xm[ym==k, 1], s=8, alpha=0.6, label=f"moon {k+1}")

axes[1].set_title("two-moons samples")
axes[1].legend(); axes[1].set_aspect('equal')

plt.tight_layout()
plt.show()
```



A single mean and covariance can only ever describe one ellipse, and the GMM's two sets of data each already follow this shape, thus their summary roughly captures where the data is. In the case of two-moons' data, this dataset creates a spiral shape for each moon, so the same summary would describe data that would just sit in the empty space between the two moons and miss the shape entirely.